

16.09.2026

Лекция 3. Рекуррентные формулы

Задача 1. Вычисление суммы

Пусть задана множество $\{x_i\}, i = \overline{1, N}$. Сумму элементов множества обозначим $S_N = \sum_{i=1}^N x_i$ и зададим алгоритм для ее нахождения.

Наивная реализация выглядит следующим образом

```

$$S_N = 0$$
for  $i = 1 : N$   
     $S_N = S_N + x_i$   
end for
```

и не предполагает использование параллельных вычислений.

Если t - время выполнения арифметической операции, то общее время выполнения алгоритма $T_1 = N \cdot t$.

Для использования параллельных вычислений можно решить эту задачу алгоритмом сдваивания.

┌

Алгоритм сдваивания ($n = 2^3 = 8$)

Этапы	e_1	e_2	e_3	e_4	e_5	e_6	e_7	e_8
1	$e_1 + e_2$		$e_3 + e_4$		$e_5 + e_6$		$e_7 + e_8$	
2	$e_1 + e_2 + e_3 + e_4$				$e_5 + e_6 + e_7 + e_8$			
3	$e_1 + e_2 + e_3 + e_4 + e_5 + e_6 + e_7 + e_8$							

Будем считать $N = 2^q$, хотя алгоритм работает и для произвольного N . Количество процессоров на каждом этапе

обозначим за p .

На 1 этапе $p = \frac{N}{2} = 2^{q-1}$ и затратили время t .

...

На k -м этапе $p = \frac{N}{k} = 2^{q-k}$ и затратили время t .

...

На $p - 1$ этапе $p = \frac{N}{4} = 2^{q-2}$ и затратили время t

На $p - 1$ этапе $p = 1$ и затратили время t .

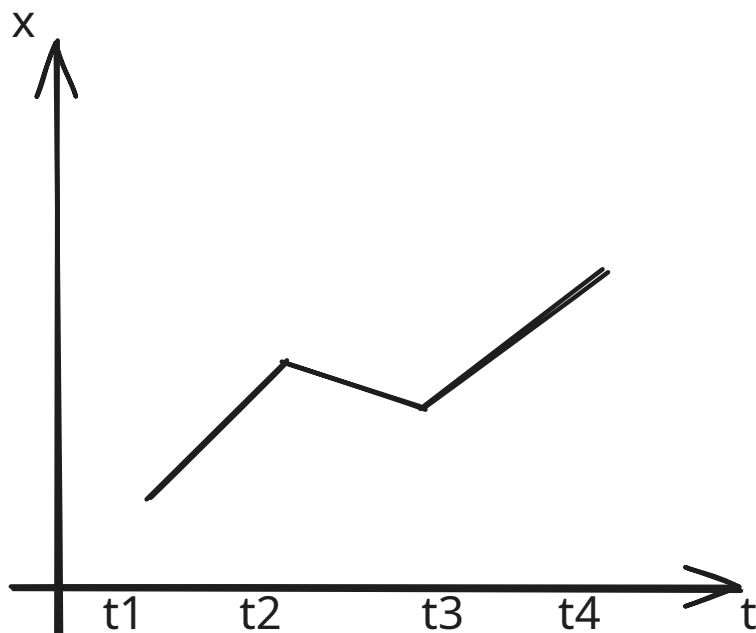
Суммарное время работы алгоритма $T_{N/2} = q \cdot t$.

$$\text{Ускорение } S_{N/2} = \frac{T_1}{T_2} = \frac{N \cdot t}{q \cdot t} = \frac{N}{q} = \frac{N}{\log N}$$

Задача 2. Вычисление частичных сумм

Пусть задана множество $\{x_i\}, i = \overline{1, N}$. Требуется найти все

$$S_k = \sum_{i=1}^k x_i$$



$$x_i = f(t_i)$$

$$y_i = \int f(t) dt$$

$$y_k = h \cdot \sum_{i=1}^k x_i, \quad h = t_{i+1} - t_i$$

Наивный алгоритм будет выглядеть так

```

 $S_k = 0, \quad k = \overline{1, N}$ 
for  $i = 2 : N$ 
     $S_i = S_{i-1} + x_i$ 
end for

```

Его время выполнения $T_i = N \cdot t$.

Каскадная схема позволяет использовать параллельные вычисления для этой задачи.

Этап	$x_1 \ (S_1^{(0)})$	$S_2^{(0)} = x_2$	$S_3^{(0)} = x_3$	$S_{n-1}^{(0)} = x_{n-1}$
1	$S_1^{(1)} = x_1$	$S_2^{(1)} = x_2 + x_1$	$S_3^{(1)} = x_3 + x_2$	$S_{n-1}^{(1)} = x_{n-1} + S_{n-2}^{(1)}$
2	$S_1^{(2)} = S_1^{(0)}$	$S_1^{(2)} = S_1^{(1)}$	$S_3^{(2)} = S_3^{(1)} + S_1^{(1)} = S_3$	$S^{(n-1)} = S_{n-1}^{(1)} + S_1^{(1)}$
3	$S^{(3)}$			

На "нулевом" этапе мы получаем 1 частичную сумму, на первом 2, на третьем 8 и т.д.

Время выполнения $T_N = q \cdot t$.

$$\text{Ускорение } S_N = \frac{T_1}{T_N} = \frac{N \cdot t}{q \cdot t} = \frac{N}{q} = \frac{N}{\log N}.$$

Алгоритм может выглядеть так

```

 $\{x_i\}_{i=1}^N, S_i^{(k)}, N = 2^q$ 
 $S^{(0)} = x$ 
for  $k = 1, q$ 
     $shift = 2^{k-1}, m = shift + 1$ 
     $S^{(k)}[1 : shift] = S^{(0)}[1 : shift]$ 
    // Цикл ниже можно распараллелить
    for  $i = m, N$ 
         $S_i^{(k)} = S_i^{(k-1)} + S_{i-shift}^{(k-1)}$ 
    end for
end for

```

Его можно доработать до такого состояния

```

 $\{x_i\}_{i=1}^N, N = 2^q$ 
 $S_{complete} = x$ 
for  $k = 1, q$ 
     $shift = 2^{k-1}, m = shift + 1$ 
    for  $i = m, N$ 
         $S_{buffer, i} = S_{complete, i} + S_{complete, i-shift}$ 
    end for
     $S_{complete}[m : N] = S_{buffer}[m : N]$ 
end for

```